

Relatório Do Projeto 1 de P.O.O

Sistema de Gestão de Áreas de Lazer

1st Lucas Araújo Campos Szuster

DELT
UFMG

Belo Horizonte, Brasil
lucasacszuster@gmail.com

2nd Arthur Coronho Seabra Eiras

DELT
UFMG

Belo Horizonte, Brasil
arthurcse@ufmg.br

3rd Caio César Araújo Guimarães

DELT
UFMG

Belo Horizonte, Brasil
caio.araujog@gmail.com

Abstract—Esse documento explica e especifica a implementação da solução de um problema muito comum presente em clubes, condomínios e quaisquer outros estabelecimentos que necessitem gerenciar suas áreas comuns. Ao longo do texto, a arquitetura do projeto será especificada, assim como suas classes com seus métodos associados e as estratégias que usamos para ter nosso produto final.

Index Terms—área, evento, interface, gerenciadora, integrar, usuário.

I. INTRODUÇÃO

A ideia do projeto foi criar um sistema para gestão de áreas de lazer, algo muito comum no cotidiano, seja em condomínios, apartamentos ou até mesmo clubes. A ideia é utilizar dessa situação como um exemplo de aplicação dos princípios da Programação Orientada a Objetos (POO) para resolver um problema prático que é o gerenciamento das áreas de lazer disponíveis entre os moradores. Assim, implementamos nosso projeto na linguagem **Python** utilizando de boas práticas aliado aos conhecimentos adquiridos na sala de aula.

II. ARQUITETURA DO PROJETO

A arquitetura do projeto se baseou em alguns pilares:

A. Princípios da POO

Alguns dos princípios fundamentais da **POO** são dados pela sigla **SOLID** que foi usada extensivamente em nosso projeto:

- **S**: Mantemos o princípio da responsabilidade única com classes distintas com escopos bem definidos.
- **O**: O princípio do aberto/fechado foi usado nos atributos e métodos das classes colocando os atributos como privados e os métodos públicos.
- **L**: É possível ver o princípio de substituição de Liskov na classe área onde suas classes filhas podem substituí-la em determinada situação sem alterar o funcionamento.
- **I**: O princípio de segregação de interfaces também foi utilizado já que todas as classes implementam métodos usados por elas
- **D**: Por fim, o princípio de inversão de dependência foi aplicado para abstrair e simplificar o problema, sendo um claro exemplo a classe área que implementa uma

abstração de alto nível e define o comportamento e escopo para suas sub-classes

B. Diagrama UML

O diagrama UML foi fundamental para a visualização e elaboração do projeto de uma maneira mais prática tornando as classes e suas relações mais claras. Apesar do diagrama ter sido modificado substancialmente, o objetivo inicial do projeto se manteve e o uso do **UML** tornou mais fácil a implementação e desenvolvimento do sistema.

C. Métodos de Engenharia de Sistemas

Nesse projeto, utilizamos um pouco dos conhecimentos adquiridos na disciplina de Processos e Métodos onde temos os métodos de Engenharia de Sistemas que são uma base para o desenvolvimento e estruturação do ciclo de vida de um sistema. Esses métodos são divididos em 4 fases sequenciais sendo elas:

1) *Análise de Requisitos*: A análise de requisitos é baseada em compreender as entradas ainda que incompletas e corrigir quando necessário. A parte chave dessa fase é a organização e interpretação das entradas de forma a compreendê-las e dar uma direção confiável ao sistema. Simplificando em uma frase, análise de requisitos é sobre clarificar, corrigir e quantificar. No nosso projeto, os requisitos demandados foram:

- O sistema precisa disponibilizar áreas específicas com restrições bem determinadas.
- Os eventos devem ser congruentes com a área de uso e ter duração e lotação máxima especificadas.
- Os usuários devem estar devidamente separados entre consumidores e administradores de eventos.
- Deve ser possível marcar e desmarcar eventos com antecedência respeitando a devida ordem temporal.
- Os eventos, datas e usuários devem estar armazenados em um banco de dados.
- O sistema deve ser capaz de gerar um relatório para a análise de dados contendo as informações de usuários, áreas e eventos com suas durações correspondentes.

2) *Definição Funcional*: A definição funcional é a fase de organização das funções do sistema, isto é quais são os blocos funcionais do sistema e como eles se relacionam entre si e em quais arranjos eles podem ser colocados de forma a beneficiar

o sistema como um todo. Em nossa implementação usamos os seguintes blocos funcionais:

- Entradas
- Processamento/Interações entre os componentes
- Saídas e dados finais

3) *Definição Física*: A definição física é a etapa responsável por traduzir o projeto funcional em *hardware* ou, no nosso caso, *software*, assim sendo possível a aplicação prática das partes do sistema, ou seja das classes e métodos do sistema de fato. Nessa etapa ocorre a integração dos vários componentes do sistema bem como uma síntese das possíveis alternativas à disposição do projeto. Dessa forma, tendo a abordagem preferida selecionada fazemos a definição da interface. Nosso projeto utilizou dessa fase como forma de expandir o escopo de atuação do sistema fazendo com que ele abranja um nível maior de cenários possíveis.

4) *Validação de Projeto*: A validação de projeto é a etapa final dos métodos e possui como objetivo a validação dos requisitos iniciais e das métricas de desempenho do sistema, portanto, podemos afirmar que essa fase é focada em testes e avaliações técnicas corrigindo erros quando necessário. Uma parte crucial dessa fase é a análise de dados onde os componentes do sistema são testados para detectar erros ou pontos fracos assim contribuindo para a integridade e cumprimento dos requisitos. No nosso projeto, utilizamos diversas entradas fictícias para simular uma demanda real de eventos para um condomínio e, alterando o tamanho da entrada e os parâmetros foi possível verificar a performance do sistema bem como descobrir qual o seu gargalo.

III. CLASSES DO PROJETO

As principais classes responsáveis pelo funcionamento do projeto são:

A. Área

A classe área é uma estrutura de dados abstrata que se ramifica em dois tipos no projeto: **Área Esportiva** e **Área Social**. Elas representam os espaços físicos nos quais os eventos acontecem. Logo, cada área possui um nome e é responsável por conter seus próprios eventos, garantindo que eles sejam marcados com um espaçamento mínimo delimitado pelo seu tipo. Para áreas esportivas, a duração de cada evento é de 50 minutos. Logo, se um usuário realizar uma marcação às 13:00 e outro às 13:25, o segundo evento não poderá ser agendado. Entretanto, se ele quiser agendar às 14:00 por exemplo, o sistema o permitirá. Cada área também possui sua quantidade máxima de pessoas, que não deve ser ultrapassada no caso de um evento. A classe também possui uma lista para restrições e outra para itens. As restrições são as especificações de limitações funcionais do espaço, como "proibido nadar", enquanto itens são objetos que podem complementar uma área, como uma "Televisão" em dada área social.

A classe área possui a implementação dos métodos que todas suas classes filhas compartilham. Mas existem também

métodos que foram implementados nas classes filhas por especializarem certos aspectos. Aqui apresentaremos os principais métodos presentes em **Área**, **Área Esportiva** e **Área Social**:

Adicionar Evento: O principal método da classe. Recebe um evento e confere se a quantidade de pessoas presente no evento é maior do que a permitida na área. Caso não seja, são realizadas checagens de instância e o método confere se o horário do evento recebido não colide com o de nenhum previamente agendado naquela área. Caso todas essas etapas forem um sucesso, o evento é marcado, adicionando-o na lista de eventos do espaço e o intervalo de tempo designado é bloqueado para marcações posteriores.

Remover Evento: Recebe o evento a ser removido e o remove da lista. O intervalo de tempo que ele ocupava agora está vazio e pode ser agendado.

Adicionar Restrição: Recebe uma restrição em forma de texto, confere se ela já existe na área associada e caso não, a adiciona na lista de restrições do espaço.

Remover Restrição: Dessa vez, a função recebe a restrição que deve ser retirada, busca por ela e caso ela exista, estando presente na lista de restrições, é removida de lá.

Como cada tipo de área visa uma demanda diferente, eles devem possuir itens permitidos para seu tipo. Logo, a verificação em cada caso é diferente, mas a lógica delas é a mesma. Cada um desses métodos é implementado na sua devida classe filha de área, mas todas compartilham o mesmo funcionamento:

Adicionar Item: O método adicionar item segue uma lógica parecida com adicionar restrição. Um item em forma de texto é recebido e o sistema checa se esse item já existe na lista de itens da área. Entretanto, além disso, cada tipo de área possui uma lista com os itens permitidos nela. Por isso, também é feita uma checagem se o item recebido está contido dentro dos permitidos e caso essa checagem também for verdadeira, ele é adicionado.

Remover Item: Para a remoção de um item, o método recebe somente o item a retirar e o busca na lista. Caso exista, é excluído.

B. Evento

A classe evento é a menor unidade do nosso sistema. A ideia é de que os eventos sejam realizados nas áreas. É por isso que elas possuem listas de eventos. Cada evento possui seu respectivo nome, uma data (dia e horário), a quantidade de participantes. Os eventos são independentes nunca acontecem no mesmo lugar ao mesmo tempo.

C. Gerenciadora

A classe gerenciadora é fundamental para o funcionamento do programa pois ela que maneja e garante que todas as interações aconteçam da maneira correta e esperada para o funcionamento do sistema. A gerenciadora é responsável pela integração das várias partes do programa, contendo a lista de usuários e uma lista de áreas. Ela é encarregada de checar qual o tipo de usuário realiza certas ações e por chamar os métodos específicos das classes terceiras. No nosso programa

a ideia é que somente usuários administradores possam adicionar e remover itens e restrições em áreas, além de criarem novos usuários. Quem realiza tal controle é gerenciadora. Ela também controla os ID's dos usuários, possuindo uma variável que é incrementada toda vez que um usuário é criado. A ideia é que todos principais processos do programa passem por ela. Por isso, seus principais métodos são:

Adicionar Restrição em Área: Recebe uma restrição e uma área. Busca então essa área e chama o método adicionar restrição dela, passando a restrição análoga.

Remover Restrição em Área: Recebe uma restrição e uma área. Busca então essa área e chama o método remover restrição dela, passando a restrição análoga para remoção.

Validar Usuário: Recebe um Usuário e uma senha e checka se ele existe. Para isso, percorre a lista de usuários presente na gerenciadora e checka se o nome do usuário E sua senha batem e estejam contidos na lista. Caso sim, retorna verdadeiro, caso contrário retorna falso.

Adicionar Usuário: Recebe como parâmetro um Usuário, checka o tipo de instância e adiciona o usuário na lista. Após isso, incrementa o ID para usuários presente na gerenciadora, garantido que os usuários tenham um ID incremental e que o próximo usuário a ser criado seja do tipo:

$$ID_{\text{novoUsuario}} = ID_{\text{ultimoUsuario}} + 1$$

Buscar Usuário por ID: Recebe um inteiro ID de usuário e procura o usuário com identificador análogo, caso exista. Se o resultado da busca não for satisffeito, é gerado um erro. Caso contrário, o método retorna o Usuário buscado.

Adicionar Evento: Recebe um ID de usuário, um evento e uma área. O propósito desse método é adicionar tanto o evento em área como em usuário. Para isso, busca a área na lista de áreas e chama o método de adicionar evento nessa área e busca o usuário na lista de usuários e chama o método de adicionar evento naquele usuário.

Remover Evento: Recebe um evento e chama o método de remoção da lista de eventos do usuário que o continha assim como o de remoção da lista de eventos da área que o hospedava.

Checkar ADM: Método que recebe um ID de usuário, busca por ele na lista análoga e confere se o dito-cujo possui a flag de administrador igual a verdadeiro ou falso. Retorna True ou False a depender do valor da bandeira.

Adicionar Área: Recebe uma área, checka sua instância e adiciona na lista de áreas.

Buscar Área por nome: Recebe uma string de nome área e busca a área associada ao nome dela na lista de áreas. Retorna a área procurada.

Gerar Relatório: Gera um relatório de marcação de evento, com todos os dados.

Adicionar Item em Área: Recebe um nome de área e um item. Realiza então a busca por essa área e chama o método de adicionar item na lista dela, passando o item recebido.

Remover Item em Área: Recebe o nome da área e do item que deve ser retirado dela. Busca então a área e chama o método de remover item dela.

D. Banco de dados

O banco de dados é responsável por carregar e armazenar os dados do programa. Ele armazena as áreas, eventos e usuários, contendo todas especificações. Por exemplo, para uma área esportiva, o banco de dados salva os atributos da área respectiva, como nome, quantidade máxima de pessoas além das específicas, sua lista de restrições e itens. Além dos seus atributos específicos como esporte jogado e se possui iluminação. Ele é carregado no início da execução e, ao fim dela, salva quaisquer alterações realizadas no meio tempo.

E. Usuário

A ideia é que hajam dois tipos de usuários: os administradores e os usuários comuns. Como no fim das contas o único fator que diferencia um administrador de um usuário comum é somente o status, não havia sentido em criar uma classe derivada de usuário só para ele, já que todos métodos e implementações seriam os mesmos. Por isso, o usuário tem uma flag **isADM** que no caso do usuário ser administrador é verdadeira e caso contrário, falsa. Cada usuário possui então um nome, id, senha e sua variável de controle se é administrador. Somente Adms podem criar novos usuários, depenndo então que o sistema já possuia, em sua inicialização inicial um adm default. Todos usuários também contém uma lista de eventos associada a ele. Assim, é que cada usuário cotenha os eventos marcados por ele, cada um deles em uma determinada área e em um horário específico, evitando, dessa forma, confusões e tornando mais fácil a organização tanto para o sistema como para quem está o usando. Os principais métodos de **Usuário** são então:

Adicionar Evento: Recebe um evento, checka sua instância e o adiciona na lista de eventos do usuário.

Remover evento: Recebe um evento a ser retirado e o remove da lista de eventos.

F. Interface Gráfica

A interface gráfica é responsável pela interação direta do usuário com o sistema. É por ela que quem usa o sistema visualiza, interage e realiza ações, como criar um novo usuário, agendar um evento, adicionar item, etc. Todas funções vistas anteriormente, são chamadas após o usuário realizar uma ação por meio da interface gráfica. O sistema foi dividido em várias telas e cada ação tem uma tela associada. Após logar no software, por exemplo o usuário tem acesso a tela de menu. Nessa tela é realizada uma checagem chamando o método em gerenciadora de conferir se user é adm. Caso seja verdade, a tela exibe, além das opções padrões do menu, as alternativas para administradores também. As telas serão mais bem explicadas em nos subtópicos da classe. Em razão disso, ela é uma classe muito extensa e possui inúmeros métodos, já que diversos métodos de gerenciadora tem que ser chamados pela interface gráfica, já que é por meio dela que o usuário

realiza suas ações. Se um usuário agenda um evento, ele interage com a interface gráfica que chama a gerenciadora que busca a área na qual o evento vai acontecer e chama tal área para que seja possível adicionar o evento na lista de eventos da área. Segue aqui uma demonstração gráfica do caminho da informação:

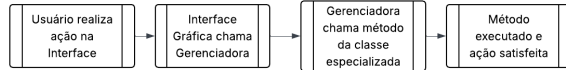


Fig. 1. Fluxo de chamadas de funções para realizar ações no sistema

Aqui então segue a lógica de funcionamento das telas, citando e explicando alguns principais métodos da **interface-Gráfica**:

As telas são modularizadas e cada uma delas está contida dentro de uma função. Elas são chamadas pelo um método **Trocar Tela** que recebe uma tela a ser trocada e chama a função associada que leva à exibição da nova página. Em cada tela há um objeto do tipo **Frame**. Ele é responsável por conter tudo que tem na página. Ele agrupa tanto os textos, quanto as caixas de opções, seleção e os botões e quaisquer outros elementos presentes na tela. Os nomes das telas também estão organizados em um enumerador, organizando e tornando mais fácil realizar comparações entre elas. Cada botão está associado a uma tela. Quando ele é apertado, é chamada a função **Trocar Tela** presente na própria interface, que recebe o nome da tela na qual o sistema deve ir, compara com as telas presentes no enumerador, e, para cada item do enumerador existe uma função de gerar tela associada. Com isso, a tela para qual o sistema deve ir é identificada e sua função análoga é chamada. Isso altera a tela vigente.

Além disso, como explicado anteriormente, a **interface-Gráfica** deve interagir com a gerenciadora para que o sistema funcione como um todo. A maneira implementada para que isso aconteça, é que a própria classe da interface possua funções associadas aos métodos de gerenciadora que chamam essas funções na classe controladora. Então se um usuário deseja adicionar um evento, a interface gráfica, estando na tela da adição, chama uma função da própria classe chamada adicionar evento que recebe os dados do utilizador pela entrada padrão que então chama o método adicionar evento na gerenciadora, que realiza seu papel de buscar e integrar os diversos setores do sistema. A ideia então é que para cada funcionalidade primária do sistema que esteja presente na gerenciadora exista uma função associada na interface gráfica que torne possível a comunicação geral do software. Ilustração:

As telas implementadas no nosso programa são as seguintes:

- **Tela de Login:** Permite que o usuário autentique sua identidade e acesse as funcionalidades do sistema por meio da inserção de seu nome de usuário e senha.
- **Tela de Menu do Usuário:** Apresenta as funcionalidades disponíveis para o usuário, adaptando as opções exibidas

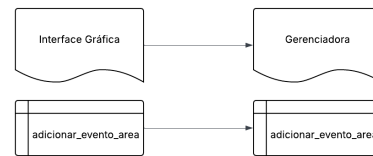


Fig. 2. Fluxo de InterfaceGráfica para Gerenciadora

de acordo com seu nível de permissão, como usuário comum ou administrador.

- **Tela de Áreas Disponíveis:** Exibe todas as áreas cadastradas no sistema, permitindo que os usuários visualizem os eventos associados a cada área e que administradores gerenciem itens e restrições vinculados a elas.
- **Tela de Eventos da Área:** Permite a visualização de todos os eventos agendados em uma área específica. Usuários com permissões adequadas, os administradores ou responsáveis pelo evento, podem realizar sua remoção.
- **Tela de Eventos Próprios:** Exibe todos os eventos agendados pelo usuário, apresentando informações como horário e data.
- **Tela de Criação de Eventos:** Permite ao usuário cadastrar novos eventos, coletando todas as informações necessárias para seu agendamento.
- **Tela de Criação de Usuários:** Disponível exclusivamente para administradores, possibilita o cadastro de novos usuários no sistema, coletando todas as informações necessárias.
- **Tela de Criação de Áreas:** Disponível exclusivamente para administradores, permite o cadastro de novas áreas no sistema, coletando todas as informações necessárias.
- **Tela de Adição de Restrições à Área:** Disponível exclusivamente para administradores, permite cadastrar novas restrições associadas a uma área específica.
- **Tela de Remoção de Restrições da Área:** Disponível exclusivamente para administradores, possibilita a remoção de restrições previamente cadastradas em uma área.
- **Tela de Adição de Itens:** Disponível exclusivamente para administradores, permite adicionar novos itens a uma área cadastrada.
- **Tela de Remoção de Itens:** Disponível exclusivamente para administradores, possibilita a remoção de itens previamente associados a uma área.

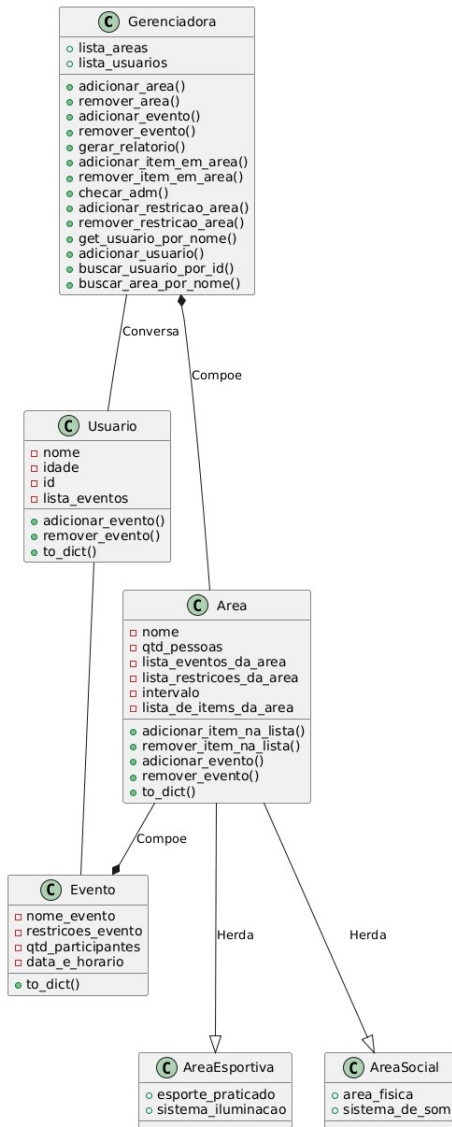
IV. CONCLUSÃO

Considerando todo o desenvolvimento do projeto e os desafios de aprendizagem relacionados a ele, podemos afirmar que ele cumpre seus objetivos, no quesito de ser capaz de resolver o problema proposto inicialmente: O gerenciamento de áreas para eventos no contexto de um condomínio ou clube utilizando os conhecimentos de Programação Orientada a Objetos e da linguagem **Python** vistos em sala de aula. Nesse sentido, considerando o contexto da disciplina, fomos capazes de analisar, projetar e implementar o projeto sob a

ótica orientada a objetos onde definimos o escopo e os objetivos primários, depois fizemos o diagrama UML de classes e, durante a implementação, aprimoramos e expandimos o escopo do projeto que em primeiro plano se restringiria a uso condominial para ser capaz de ser usado por clubes e empresas. Durante a implementação do código, os pontos mais complexos foram relacionados a organização das listas de usuários e evento, além da implementação da interface gráfica.

V. APÊNCIDE

Segue aqui o diagrama UML que auxiliou o grupo durante o desenvolvimento do projeto:



REFERENCES

- [1] Real, Luiza, Slides da Disciplina de Análise, Projeto e Programação Orientada a Objetos. 2026.
- [2] Batista, André, Liddy, Ana, Slides da Disciplina de Processos e Métodos em Engenharia de Sistemas. 2026.